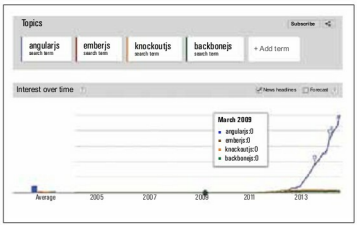

Figure 1: Report from Google Trends

from models. This helps an SPA in redrawing any part of the UI without requiring a server round trip to retrieve HTML. When the data is updated, its view is notified and the altered data is produced in the view.

Data binding

AngularJS provides you with two-way binding between the model variables and HTML elements. One-way binding would mean a one-way relation between the two—when the model variables are updated, so are the values in the HTML elements; but not the other way around. Let's understand two-way binding by looking at an example:

```
<html ng-app >
<head>
    <script src="http://ajax.googleapis.com/ajax/libs/
angularjs/1.0.7/angular.min.js">
    </script>
</head>
<body ng-init = "yourtext = 'Data binding is cool!' ">
    Enter your text: <input type="text" ng-model =
"yourtext" />
    <strong>You entered :</strong> {{yourtext}}
 </body>
 </html>
```

The model variable *yourtext* is bound to the HTML input element. Whenever you change the value in the input box, *yourtext* gets updated. Also, the value of the HTML input box is initialised to that of the *yourtext* variable.

Directives

In the above example, many words like *ng-app*, *ng-init* and *ng-model* may have struck you as odd. Well, these are attributes that represent directives - *ngApp*, *ngInit* and *ngModel*, respectively. As described in the official AngularJS developer guide, "Directives are markers on a DOM element (such as an attribute, element name, comment or CSS class) that tell AngularJS's HTML compiler (*$compile*) to attach a specified behaviour to that DOM element." Let's look into the purpose of some common directives.

ngApp: This directive bootstraps your angular application and considers the HTML element in which the attribute is specified to be the root element of Angular. In the above example, the entire HTML page becomes an angular application, since the 'ng-app' attribute is given to the *<html>* tag. If it was given to the *<body>* tag, the body alone becomes the root element. Or you could create your own Angular module and let that be the root of your application. An AngularJS module might consist of controllers, services, directives, etc. To create a new module, use the following commands:

```
var moduleName = angular.module( 'moduleName ', [ ] );
// The array is a list of modules our module depends on
```

Also, remember to initialise your *ng-app* attribute to *moduleName*. For instance,

```
<html ng-app = "moduleName" >
```

ngModel: The purpose of this directive is to bind the view with the model. For instance,

```
<input type = "text" ng-model = "sometext" />
<p> Your text: {{ sometext }}</p>
```

Here, the model 'sometext' is bound (two-way) to the view. The double curly braces will notify Angular to put the value of 'sometext' in its place.

ngClick: How this directive functions is similar to that of the *onclick* event of Javascript.

```
<button ng-click="mul = mul * 2" ng-init="mul = 1"> Multiply
with 2 </button>
 After multiplying : {{mul}}
```

Whenever the button is clicked, 'mul' gets multiplied by two.

Filters

A filter helps you in modifying the output to your view. You can subject your expression to any kind of constraints to give out the desired output. The format is:

```
 {{ expression | filter }}
```

You can filter the output of filter1 again with filter2, using the following format:

```
{{ expression | filter1 | filter2 }}
```

The following code filters the members of the people array using the name as the criteria: